

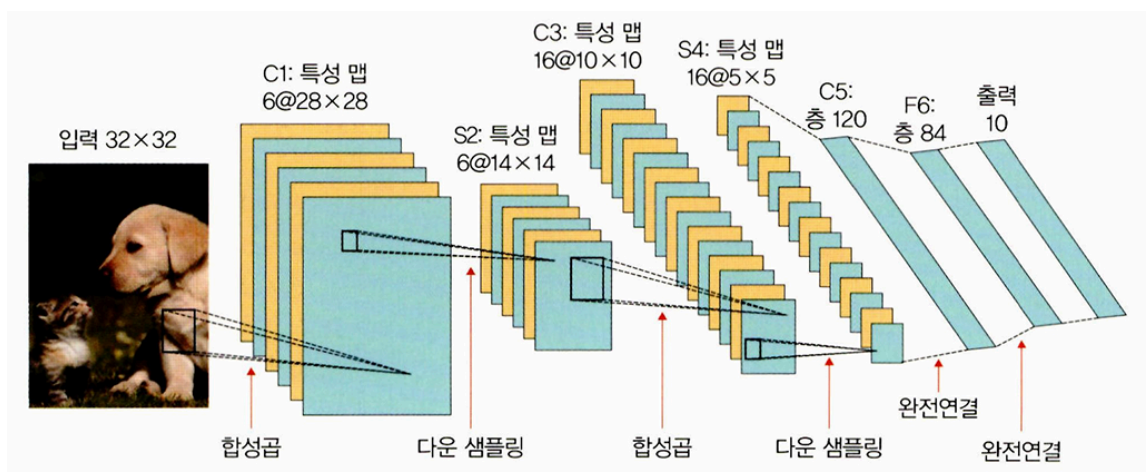
Chapter 6. 합성곱 신경망 II - Part 1(~6.1.4)

6.1 이미지 분류를 위한 신경망

6.1.1 LeNet-5

개요

- 수표에 쓴 손글씨 숫자를 인식하는 딥러닝 구조 LeNet-5 발표 → 현재 CNN의 초석
- convolutional과 sub-sampling을 반복, 마지막에 완전연결층에서 분류 수행
- LeNet-5의 과정(예시)

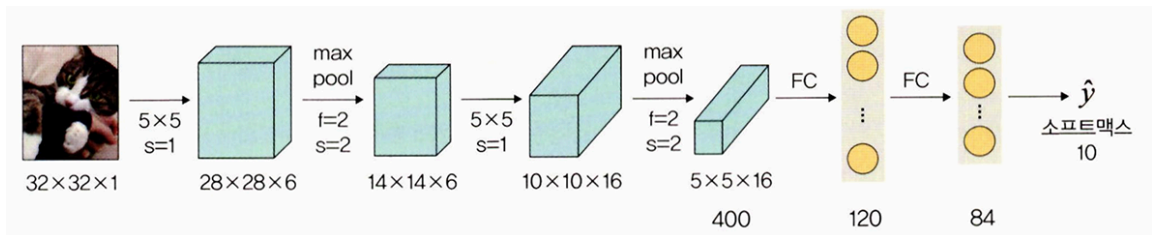


◦ C는 합성곱층, S는 풀링층, F는 완전연결층을 의미

1. C1: 5x5 합성곱 연산 → 28x28 크기의 특성 맵 6개 생성
2. S2: 다운 샘플링 → 특성 맵 크기를 14x14로 줄임
3. C3: 5x5 합성곱 연산 → 10x10 크기의 특성 맵 16개 생성
4. S4: 다운 샘플링 → 특성 맵 크기를 5x5로 줄임
5. C5: 5x5 합성 연산 → 1x1 크기의 특성 맵 120개 생성
6. F6: 완전연결층으로 C5의 결과를 유닛 84개에 연결

☀ LeNet-5 예제 구현을 해보자!

- 구현할 신경망



- 예제 신경망에 대한 상세 설명

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	32×32	—	—	—
합성곱층	6	28×28	5×5	1	렐루(ReLU)
최대 풀링층	6	14×14	2×2	2	—
합성곱층	16	10×10	5×5	1	렐루(ReLU)
최대 풀링층	16	5×5	2×2	2	—
완전연결층	—	120	—	—	렐루(ReLU)
완전연결층	—	84	—	—	렐루(ReLU)
완전연결층	—	2	—	—	소프트맥스(softmax)

#예제 진행 전 tqdm라이브러리 설치-진행 상태를 바(bar) 형태로 가시화

```
#라이브러리는 두 가지 방법으로 설치 가능(아나콘다)
!pip install --user tqdm
!pip install -c conda-forge tqdm
```

#코드 6-1. 필요한 라이브러리 호출

```
#코드 6-1. 필요한 라이브러리 호출
import torch
import torchvision
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms #이미지 변환(전처리) 기능을 제공하는 라이브러리
```

```

from torch.autograd import Variable
from torch import optim #경사 하강법을 이용하여 가중치를 구하기 위한 옵티마이저 라이브러리
import torch.nn as nn
import torch.nn.functional as F
import os #파일 경로에 대한 함수들을 제공
import cv2
from PIL import Image
from tqdm import tqdm_notebook as tqdm #진행 상황을 가시적으로 표현해주는 데,
                                     #특히 모델의 학습 경과를 확인하고 싶을 때 사용하는 라이브러리
import random
from matplotlib import pyplot as plt

#파이토치는 텐서플로와 다르게 GPU를 자동으로 할당X,GPU 할당을 모델과 데이터에 선언
#단 이장에서는 CPU사용
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

```

#코드 6-2. 이미지 데이터셋 전처리

- 모델 학습에 필요한 데이터셋의 전처리(예: 텐서 변환)가 필요
- torchvision 라이브러리를 이용하여 이미지 전처리

```

train_transforms = transforms.Compose([ #㉑
    transforms.RandomResizedCrop(resize, scale=(0.5, 1.0)), #㉒
    transforms.RandomHorizontalFlip(), #㉓
    transforms.ToTensor(), #㉔
    transforms.Normalize(mean,std) #㉕
])

```

- ㉑ transforms.Compose: 이미지를 변형할 수 있는 방식들의 묶음
- ㉒ transforms.RandomResizedCrop: 입력 이미지를 주어진 크기로 조정, scale은 원래 이미지를 임의의 크기(0.5~1.0(50%~100%))만큼 면적을 무작위로 자른다는 의미

- ㉓ transforms.RandomHorizontalFlip: 주어진 확률로 이미지를 수평 반전, 기본 값 0.5
- ㉔ transforms.ToTensor: 효율적인 연산을 위해 torch.FloatTensor배열로 바꾸는 메소드
- ㉕ transforms.Normalize: ImageNet 데이터의 각 채널별 평균과 표준편차에 맞게 정규화
 - OpenCV를 사용해 이미지를 읽어온다면 RGB이미지가 아닌 BGR이미지
- __call__함수: 클래스를 호출하는 메서드

```
def __call__(self, img, phase):
```

- 클래스에 __call__함수가 있을 경우 클래스 객체 자체 호출 시 __call__함수의 리턴 값 반환

#코드 6-3. 이미지 데이터셋을 불러온 후 훈련, 검증, 테스트로 분리

- 훈련용으로 400개 이미지, 검증용으로 92개 이미지, 테스트용으로 10개의 이미지 사용
- 고양이 이미지 데이터 불러오기

```
cat_images_filepaths = sorted([os.path.join(cat_directory, f) #㉑, ㉒
                               for f in os.listdir(cat_directory)]) #㉓
```

- ㉑ sorted: 데이터를 정렬된 리스트로 만들어 반환
- ㉒ os.path.join: 경로와 파일명을 결합하거나 분할된 경로를 하나로 합칠 때 사용
- ㉓ os.listdir: 지정한 디렉터리 내 모든 파일의 리스트를 반환

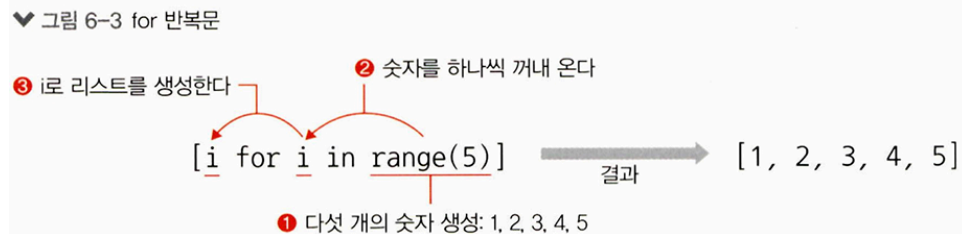
 경로를 합치는 다른 방식

```
import os
list_path = ['C:\\', 'Temp', 'user']
folder_path = os.path.join(*list_path)
folder_path
# 실행 결과: 'C:\\Temp\\user'
```

- images_filepaths에서 이미지 파일 불러오기

```
correct_images_filepaths = [i for i in images_filepaths #㉠, ㉢
                             if cv2.imread(i) is not None] #㉡
```

- ㉠ for 반복문을 이용하여 가져온 데이터에 대해 i를 이용하여 리스트로 만들



- ㉢ for 반복문을 이용하여 images_filepaths에서 이미지 데이터 검색
- ㉡ cv2.imread(): 모든 이미지 데이터를 읽어옴(더 이상 데이터 찾을 수 없을 때까지)
- 넘파이 random(): 임의의 난수 생성

```
random.seed(42)
```

- 난수 생성을 위해 시드 값 사용
- 모듈이 호출될 때마다 임의의 난수가 재생성 but 특정 시드 값 부여 시 상태 저장

#코드 6-4. 테스트 데이터셋 이미지 확인 함수

- cv2.cvtColor: 이미지의 색상을 변경

```
image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

- 첫 번째 파라미터: 입력 이미지
- 두 번째 파라미터: 변환할 이미지 색상 지정, BGR 채널 이미지를 RGB로 변경
- 이미지 전체 경로 정규화 및 분할

```
true_label = os.path.normpath(image_filepath).split(os.sep)[-2]
```

- os.path.normpath: 경로명 정규화
- split(os.sep): 경로를 / 혹은 \를 기준으로 분할할 때 사용

- predicted_label

```
predicted_label = predicted_labels[i] if predicted_labels else true_label
```

- predicted_label 값이 있으면 그 값을 predicted_label로 사용, 없다면 true_label 사용

#코드 6-6. 이미지 데이터셋 클래스 정의

- __init__에서 전체 데이터를 읽어와 경로 저장, __getitem__메서드에서 이미지를 읽어옴
- 개와 고양이 이미지가 포함된 데이터에서 이들을 예측
 - 고양이: 0, 개: 1
- 이미지 데이터에 대한 레이블값(dog, cat)을 가져오기

```
label = img_path.split('/')[-1].split('.')[0]
```

- img_path: 이미지가 위치한 전체 경로를 보여줌
- img_path.split('/')[-1]: 이미지 전체 경로에서 '/' 제거
- split('.')[0]: 위의 결과에서 '.'을 제거

#코드 6-8. 이미지 데이터셋 정의

- 실행 결과(훈련 데이터의 크기와 레이블에 대한 출력)

```
torch.Size([3, 224, 224])
0
```

#코드 6-9. 데이터로더 정의

- 파이토치의 데이터로더는 배치 관리 담당

```
train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
```

- 첫 번째 파라미터: 데이터를 불러오기 위한 데이터셋
- `batch_size`: 한 번에 메모리로 불러올 데이터 크기
- `shuffle`: 메모리로 데이터를 가져올 때 임의로 섞어서 가져오게 함
- 실행결과(데이터셋의 크기와 레이블)

```
torch.Size([32, 3, 224, 224])
tensor([0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1,
        1, 0, 1, 1, 1, 1, 0, 0])
```

#코드 6-10. 모델의 네트워크 클래스

- 네트워크의 각 부분들을 통과할 때마다 입력과 출력의 형태가 바뀜
 - Conv2d 계층에서의 출력 크기 구하는 공식

$$\text{출력 크기} = (W - F + 2P) / S + 1$$

- W : 입력 데이터의 크기(input_volume_size)
- F : 커널 크기(kernel_size)
- P : 패딩 크기(padding_size)
- S : 스트라이드(strides)

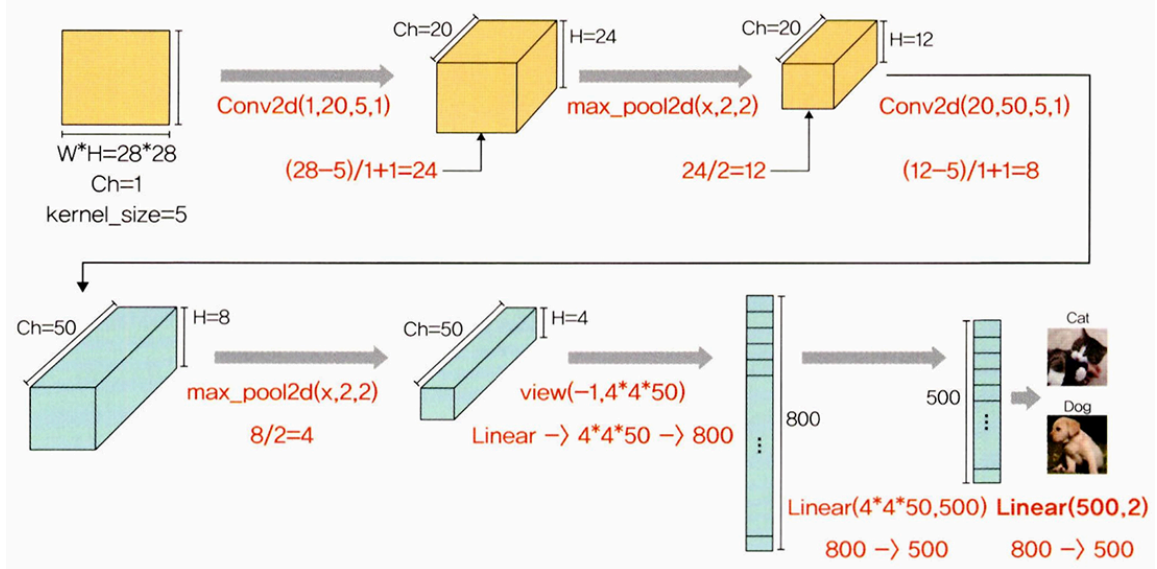
- MaxPool2d 계층에서의 출력 크기 구하는 공식

$$\text{출력 크기} = IF / F$$

- IF : 입력 필터의 크기(input_filter_size, 또한 바로 앞의 Conv2d의 출력 크기이기도 합니다)
- F : 커널 크기(kernel_size)

- 각 계층에 대한 결과

▼ 그림 6-5 합성곱층과 풀링층의 크기(형태) 계산



#코드 6-11. 모델 객체 생성 - LeNet()을 model이라는 이름으로 객체 생성

- 출력 결과가 한눈에 들어오지 않을 때는 "torchsummary"라이브러리 사용

```
!pip install torchsummary
```

#코드 6-12. torchsummary 라이브러리를 이용한 모델의 네트워크 구조 확인

- summary를 통한 모델의 네트워크 관련 정보 확인

```
summary(model, input_size=(3,224,224))
```

- 첫 번째 파라미터: 모델의 네트워크
- 두 번째 파라미터: 입력 값(채널, 너비, 높이)
- 네트워크 내 파라미터 수, 구조를 이해하기 쉬움
- 출력정보: 총 파라미터 수, 입력 크기, 네트워크 총 크기 등

#코드 6-14. 옵티마이저와 손실 함수 정의

- 경사하강법으로 모멘텀 SGD 사용

*모멘텀 SGD: 매번 기울기를 구하지만 가중치를 수정하기 전에 이전 수정 방향(+, -)를 참고하여 같은 방향으로 일정한 비율만 수정되게 하는 방법


```
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
```

- 첫 번째 파라미터: 업데이트하고자 하는 파라미터-가중치와 바이어스
- lr: 가중치를 변경할 때 얼마나 크게 변경할지 결정
- momentum: SGD를 적절한 방향으로 가속화하며 흔들림을 줄여주는 매개변수

#코드 6-16. 모델 학습 함수 정의

- 학습 용도: model.train() 사용
- 오차와 입력을 곱하는 이유는?

```
epoch_loss += loss.item() * inputs.size(0)
```

- 손실 함수의 reduction 파라미터의 기본값은 'mean' → 정답과 예측 값의 오차를 구한 후 그 값들의 평균을 반환
⇒ 손실 함수 특성상 전체 오차를 배치 크기로 나눔으로써 평균을 반환하기 때문에 epoch_loss를 계산하는 동안 loss.item()과 inputs.size(0)을 곱한다

#코드 6-17. 모델 학습

- 실행 결과

```
Training complete in 7m 44s  
Best val Acc: 0.6087
```

- 모델 학습 결과 최고 약 60% 정확도
- 데이터셋을 늘린다면 더 좋은 성능 기대

#코드 6-18. 모델 테스트를 위한 함수 정의

- 테스트 용도: model.eval() 사용
- torch.unsqueeze: 텐서에 차원 추가 시 사용, (0)은 차원이 추가될 위치

```
img = img.unsqueeze(0)
```

- 소프트맥스: 지정된 차원을 따라 텐서의 요소가 (0,1) 범위에 있고 합계가 1이 되도록 조정

```
preds = F.softmax(outputs, dim=1)[:1].tolist()
```

- outputs에 softmax를 적용하여 각 행의 값이 1이 되도록 함
- 위의 값 중 모든 행(:)에서 두 번째 칼럼(1번째 인덱스)을 가져옴
- .tolist(): 배열을 리스트 형태로 변환

#코드 6-19. 테스트 데이터셋의 예측 결과 호출

- 예측 결과 레이블이 0.5보다 크면 개, 0.5보다 작으면 고양이를 의미

#코드 6-20. 테스트 데이터셋 이미지를 출력하기 위한 함수 정의

#코드 6-21. 테스트 데이터셋 예측 결과 이미지 출력

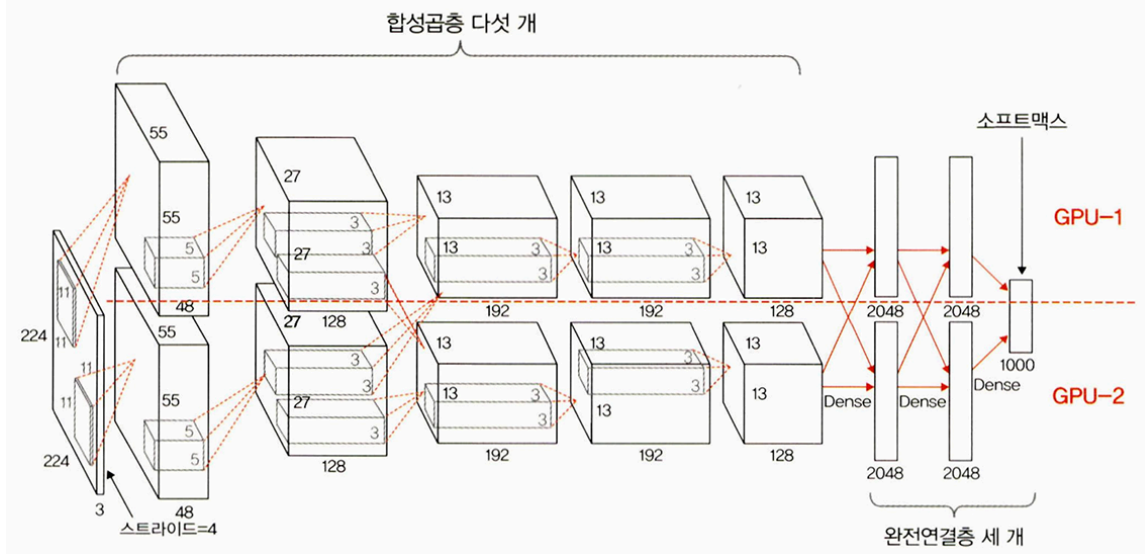
- 예측력이 좋지 않음.
- 데이터 전체를 사용하게 되면 성능 향상 기대

6.1.2 AlexNet

개요

- AlexNet 구조

♥ 그림 6-9 AlexNet 구조

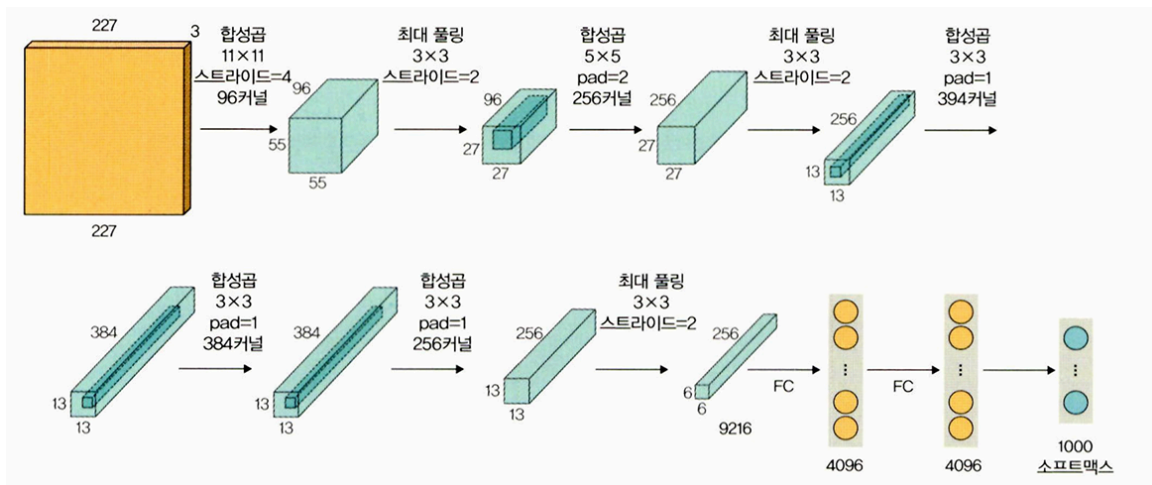


- 합성곱층 5개, 완전연결층 3개로 구성
- 맨 마지막 완전연결층은 카테고리 1000개를 분류하기 위해 소프트맥스 활성화 함수 사용
- AlexNet 구조 상세
 - 합성곱층의 활성화 함수는 ReLU

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	227×227	—	—	—
합성곱층	96	55×55	11×11	4	렐루(ReLU)
최대 풀링층	96	27×27	3×3	2	—
합성곱층	256	27×27	5×5	1	렐루(ReLU)
최대 풀링층	256	13×13	3×3	2	—
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	256	13×13	3×3	1	렐루(ReLU)
최대 풀링층	256	6×6	3×3	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

AlexNet 예제 구현을 해보자!

AlexNet 예제 네트워크



(데이터 불러오기, 전처리 과정 등은 위의 LeNet-5와 거의 똑같아서 과정 설명 생략)

#코드 6-28. 데이터셋을 메모리로 불러옴

- 실행 결과(데이터로더에서 불러온 훈련 이미지에 대한 크기와 레이블 출력)

```
torch.Size([32, 3, 226, 226])
tensor([0, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1,
        1, 0, 0, 1, 0, 1, 1, 1])
```

#코드 6-29. AlexNet 모델 네트워크 정의

- 합성곱 + 활성화함수 + 풀링이 다섯 번 반복 후 두 개의 완전연결층과 출력층으로 구성
- ReLU 활성화 함수에서 inplace

```
nn.ReLU(inplace=True),
```

- inplace 연산: 기존 값을 연산 결과값으로 대체함으로써 기존 값들을 무시하겠다는 의미

- nn.AdaptiveAvgPool2d

```
self.avgpool = nn.AdaptiveAvgPool2d((6,6))
```

◦ AvgPool2d Vs. AdaptivePool2d

- AvgPool2d

풀링에 대한 커널 및 스트라이드 크기를 정해야 동작

$$H_{out} = \left\lceil \frac{H_{in} + 2 \times padding[0] - kernel_size[0]}{stride[0]} + 1 \right\rceil$$
$$W_{out} = \left\lceil \frac{W_{in} + 2 \times padding[1] - kernel_size[1]}{stride[1]} + 1 \right\rceil$$

- AdaptivePool2d

풀링 작업이 끝날 때 필요한 출력 크기를 정의(출력에 대한 크기만 지정)

입력 크기에 변동이 있고 CNN 위쪽에 완전연결층을 사용하는 경우에 유용

(이후 진행되는 모델 학습과 예측 과정도 LeNet-5과 거의 동일)

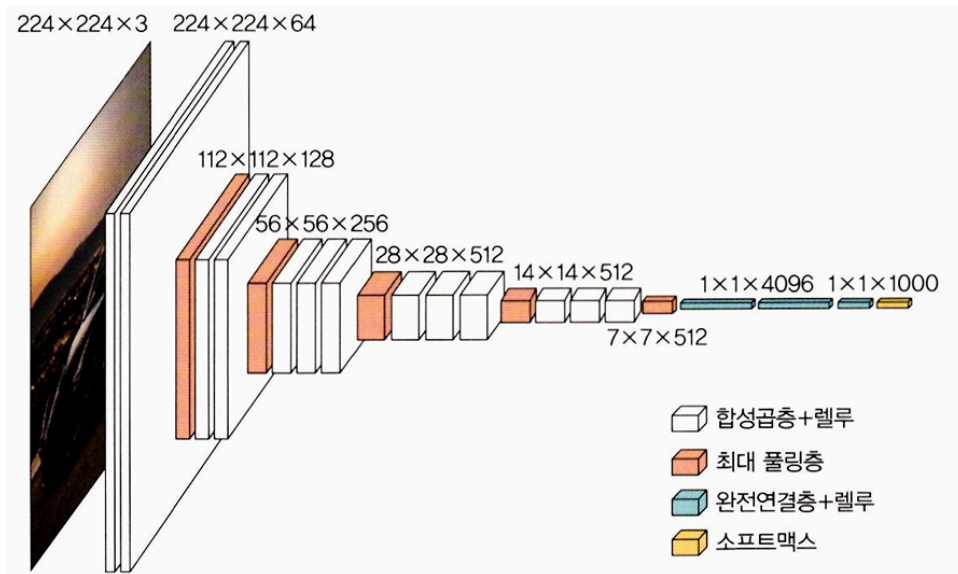
#코드 6-38. 예측 결과에 대해 이미지와 함께 출력

- LeNet-5 모델과 마찬가지로 예측 결과가 좋지 않음
- 데이터셋이 많아진다면 좋은 성능 기대

6.1.3 VGGNet

개요

- 합성곱층의 파라미터 수를 줄이고 훈련 시간을 개선하고자 탄생
- 네트워크를 깊게 만드는 것이 성능에 어떤 영향을 미치는지 확인하고자 만들어짐
- 합성곱층에서 사용하는 필터/커널의 크기를 가장 작은 3x3으로 고정
- 네트워크 계층의 개수에 따라 여러 유형이 존재(VGG16, VGG19 등)
- VGG 16
 - 합성곱 커널의 크기는 3x3, 최대 풀링 커널의 크기는 2x2, 스트라이드는 2
 - 64개의 224x224 특성 맵 생성
 - 마지막 16번째 계층을 제외하고는 모든 ReLU 함수 적용
 - VGG16 구조



◦ VGG16 구조 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	224×224	—	—	—
합성곱층	64	224×224	3×3	1	렐루(ReLU)
합성곱층	64	224×224	3×3	1	렐루(ReLU)
최대 풀링층	64	112×112	2×2	2	—
합성곱층	128	112×112	3×3	1	렐루(ReLU)
합성곱층	128	112×112	3×3	1	렐루(ReLU)
최대 풀링층	128	56×56	2×2	2	—
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
최대 풀링층	256	28×28	2×2	2	—
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
최대 풀링층	512	14×14	2×2	2	—

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
최대 풀링층	512	7×7	2×2	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

 VGGNet 예제 구현을 해보자!

#6-39. 필요한 라이브러리 호출

#코드 6-39. 필요한 라이브러리 호출

```
import copy #①
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
import torchvision.transforms as transforms
import torchvision.datasets as Datasets

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

- ① copy: 객체 복사를 위해 사용



shallow copy Vs. deep copy

- 단순 객체 복사

```
original = [1,2,3]
copy_o = original
print(copy_o)
copy_o[2] = 10
print(copy_o)
print(original)
```

- 실행결과: 원래 값인 original의 3도 10으로 바뀜

```
[1, 2, 3]
[1, 2, 10]
[1, 2, 10]
```

- shallow copy: copy.copy()

```
import copy
original = [[1,2],3]
copy_o = copy.copy(original)
print(copy_o)
copy_o[0] = 100
print(copy_o)
print(original)

append = copy.copy(original)
append[0].append(4)
print(append)
print(original)
```

- 실행결과: [1,2] 값을 100으로 변경 시 copy_o만 바뀜 but, [1,2]에 4를 추가했을 땐 original과 copy_o 모두 반영


```
[[1, 2], 3]
[100, 3]
[[1, 2], 3]
[[1, 2, 4], 3]
[[1, 2, 4], 3]
```

- deep copy : `copy.deepcopy()`

```
import
original = [[1,2],3]
copy_o = copy.deepcopy(original)
print(copy_o)
copy_o[0] = 100
print(copy_o)
print(original)

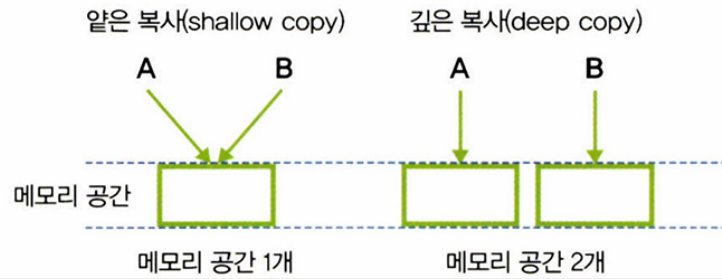
append = copy.deepcopy(original)
append[0].append(4)
print(append)
print(original)
```

- 실행결과: shallow copy와 달리 두 가지 경우 다 `copy_o`만 변경

```
[[1, 2], 3]
[100, 3]
[[1, 2], 3]
[[1, 2, 4], 3]
[[1, 2], 3]
```

- 단순 복사는 완전히 동일히 복사, 얇은 복사와 깊은 복사는 원리 차이

♥ 그림 6-14 얕은 복사와 깊은 복사



#코드 6-40. VGG모델 정의

#코드 6-41. 모델 유형 정의

- VGG11, VGG13, VGG16, VGG19 모델의 계층 정리

#8(합성곱층)+3(풀링층)=11(전체계층)=VGG11

vgg11_config = [64,'M',128,'M',256,256,'M',512,512,'M',512,512,'M']

#10(합성곱층)+3(풀링층)=13(전체계층)=VGG13

vgg13_config=[64,64,'M',128,128,'M',256,256,'M',512,512,'M',512,512,'M']

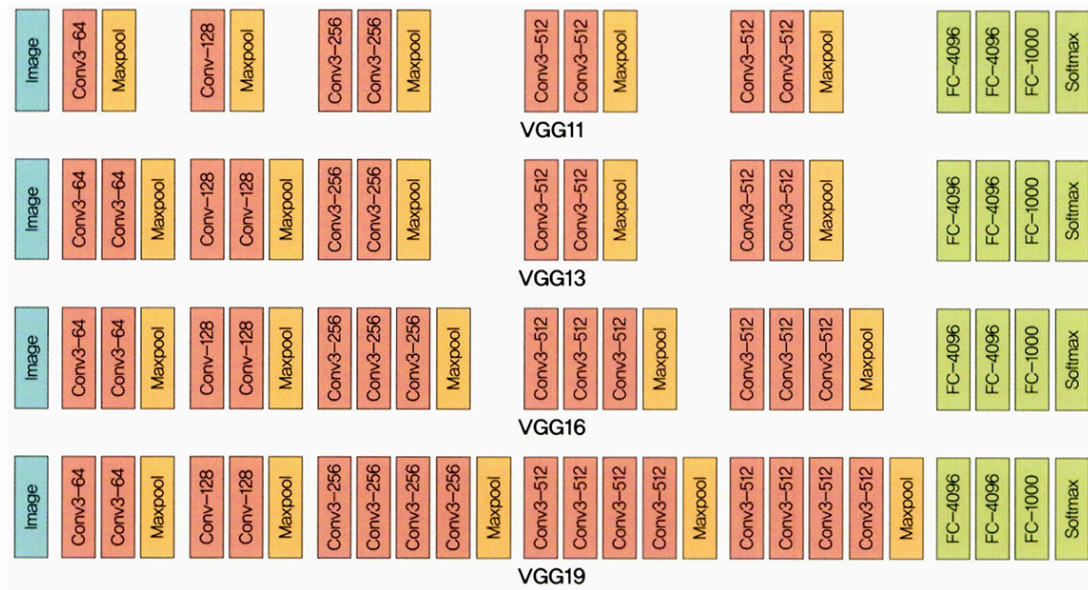
#13(합성곱층)+3(풀링층)=16(전체계층)=VGG16

vgg16_config=[64,64,'M',128,128,'M',256,256,256,'M',512,512,512,'M',512,512,'M']

#16(합성곱층)+3(풀링층)=19(전체계층)=VGG19

vgg19_config = [64,64,'M',128,128,'M',256,256,256,256,'M',512,512,512,512,'M',512,512,512,512,'M']

- 숫자는 Conv2d 수행의 의미, M은 최대 풀링 수행의 의미
- 출력 채널이 다음 계층이 입력 채널
- VGG11, VGG13, VGG16, VGG19에 대한 네트워크 그림으로 정리



#코드 6-42. VGG 계층 정의

- 조건문 정의

```
assert c == 'M' or isinstance(c,int)
```

- assert c == 'M' : 가정 설정문이라고 불리는 assert - 뒤의 조건이 True가 아닐 시 예러
- isinstance: 주어진 조건이 True인지 판단
- 따라서, c가 'M'이 아니거나 int가 아니라면 오류 발생

#6-43. 모델 계층 생성

- get_vgg_layers() 함수를 호출하여 모델의 계층 생성
- 배치 정규화에 대한 계층도 추가

```
vgg11_layers = get_vgg_layers(vgg11_config, batch_norm=True)
```

- batch_norm: 데이터의 평균을 0, 표준편차를 1로 분포 시키는 것

#코드 6-45. VGG11 전체에 대한 네트워크

- 출력 결과: 앞에서 정의한 vgg11_layers, 완전연결층과 출력층 부분이 합쳐짐

VGG 사전 훈련 모델

대용량의 이미지 데이터로 학습 시켜놓고, 최상의 상태로 튜닝을 거쳐 모든 사람이 사용할 수 있도록 공유한 사전 훈련 모델

#코드 6-46. VGG 사전 훈련된 모델 사용

- 배치 정규화가 적용된 사전 훈련된 VGG11 모델 사용

```
pretrained_model = models.vgg11_bn(pretrained=True)
```

- vgg11_bn: VGG11 기본 모델에 배치 정규화가 적용된 모델을 사용
- pretrained을 True로 설정하면 사전 훈련된 모델을 사용

#코드 6-47. 이미지 데이터 전처리

- transforms.Compose 파라미터 살펴보기

```
train_transforms = transforms.Compose([
    transforms.Resize((256,256)),
    transforms.RandomRotation(5),
    transforms.RandomHorizontalFlip(0.5),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485,0.456,0.406],std=[0.229,0.224,
0.225])
])
```

- transform.Resize: 이미지를 주어진 크기(256x256)로 재조정
- transforms.RandomRotation: 5도 이하로 이미지 회전

#코드 6-48. ImageFolder를 이용하여 데이터셋 불러오기



Imager Folder는 언제 사용하면 좋은가?

- 계층적인 폴더 구조를 갖고 있는 데이터셋을 불러올 때 사용

▼ 그림 6-16 './chap06/data/catanddog/train' 위치의 하위 폴더

☐ 이름	유형
 Cat	파일 폴더
 Dog	파일 폴더

- train 폴더 하위에 Cat, Dog 폴더가 계층적으로 위치

#코드 6-49. 훈련과 검증 데이터 분할

- 파이썬의 random_split()
데이터셋 단계에서 진행해야 함

```
train_data, valid_data = data.random_split(
    train_dataset,[n_train_examples,n_valid_examples])
```

- 첫 번째 파라미터: 분할에 사용될 데이터셋
- 두 번째 파라미터: 훈련과 검증 데이터셋의 크기 지정

#코드 6-54. 모델 정확도 측정 함수

- correct 변수: 예측이 정답과 일치하는 경우 그 개수의 합 저장

```
correct = top_pred.eq(y.view_as(top_pred)).sum()
```

- eq: equal의 약자로 서로 같은지 비교
- view_as(other): other의 텐서 크기 사용
- sum(): 합계 구하기, 예측과 정답이 일치하는 것들의 개수 합산

#코드 6-58. 모델 학습

- 실행 결과

```
Epoch: 05 | Epoch Time: 12m 1s
      Train Loss: 0.696 | Train Acc: 51.67%
      Val. Loss: 0.686 | Val. Acc: 54.72%
```

- 성능이 좋지 않음 → 데이터셋의 이미지 수가 적고 에포크도 적게 설정 되었음

#코드 6-59. 테스트 데이터셋을 이용한 모델 성능 측정

- 실행 결과, 결과 좋지 않음

#코드 6-60. 테스트 데이터셋을 이용한 모델의 예측 확인 함수

- `argmax`: 배열에서 가장 큰 값의 인덱스를 찾을 때 사용

```
top_pred = y_prob.argmax(1,keepdim=True)
```

- 첫 번째 파라미터: 행(`axis=0`) 또는 열(`axis=1`)을 따라 가장 큰 값의 색인을 찾음
- `keepdim`: `True`의 경우 출력 텐서를 입력과 동일한 크기로 유지
- `torch.cat`: 텐서 연결 시 사용

```
images = torch.cat(images, dim=0)
```



차원과 torch.cat

- 차원

♥ 그림 6-17 차원(dim)

	Col1	Col2	Col3
Row1			
Row2			
Row3			

axis=0(dim=0) (vertical arrow pointing down from Col1)

axis=1(dim=1) (horizontal arrow pointing right from Row1)

dim=0은 행, dim=1은 열을 의미

- torch.cat

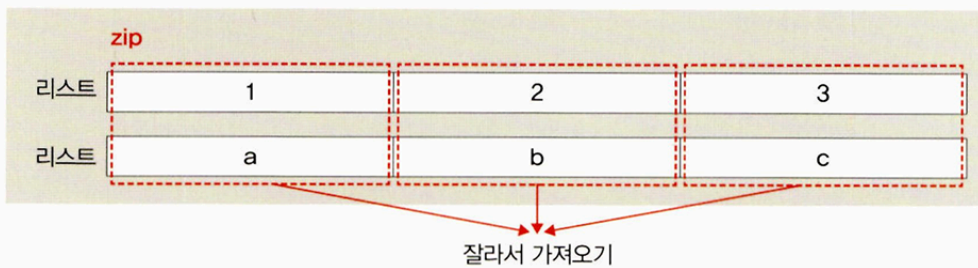
```
import torch
x = torch.Tensor([[1,2,3],[2,3,4]])
y = torch.Tensor([[4,5,6],[5,6,7]])
print(torch.cat([x],dim=0) #행을 기준으로 x를 이어붙임
print('-----')
print(torch.cat([x,y])) #단순하게 x와 y결합, (4x3)형태
print('-----')
print(torch.cat([x,y],dim=0)) #행을 기준으로 x와 y를 이어붙임, (4x3)
print('-----')
print(torch.cat([x,y], dim=1)) #열을 기준으로 x와 y를 이어붙임, (2x6)
```

#코드 6-61. 예측 중에서 정확하게 예측한 것을 추출

- zip(): 여러 개의 리스트(혹은 튜플)를 합쳐서 새로운 튜플 타입으로 반환

```
for image, label, prob, correct in zip(images,labels,probs,corrects):
```

▼ 그림 6-18 zip()의 원리



- `sort()`: 데이터 정렬

```
correct_examples.sort(reverse=True, key=lambda x:torch.max(x[2], dim=0).values)
```

- `reverse`: 내림차순 정렬
- `key`: 데이터 정렬 시, `key`값을 갖고 정렬하면 기본값은 오름차순

#코드 6-62. 이미지 출력을 위한 전처리

- `torch.add`: 더하는 메서드

```
image.add_(-image_min).div_(image_max - image_min + 1e-5)
```

- `torch.add`와 `torch.add_`의 차이
메서드에 `_` 표시: 기존의 메모리 공간에 있는 값을 새로운 값으로 대체

#코드 6-63. 모델이 정확하게 예측한 이미지 출력 함수

- `image.permute`: 축 변경 시 사용

```
image = image.permute(1, 2, 0)
```

#코드 6-64. 예측 결과 이미지 출력

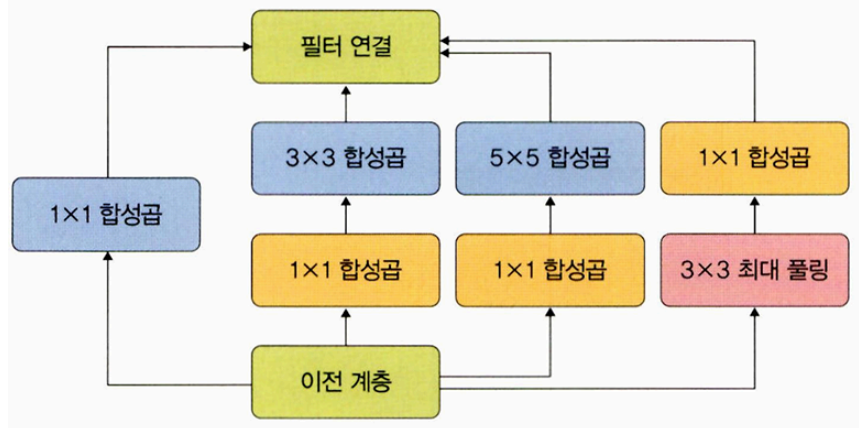
- 정확도가 높지 않음, 데이터셋 늘려주기

6.1.4 GoogLeNet

주어진 하드웨어 자원을 최대한 효율적으로 이용하면서 학습 능력 극대화하는 깊고 넓은 신경망

- inception 모듈 추가: 깊고 넓은 신경망 생성을 위해
 - 효율적으로 추출하기 위해 1x1, 3x3, 5x5의 합성곱 연산을 각각 수행
 - sparse connectivity 적용 : 관련성이 높은 노드끼리만 연결하는 방법

▼ 그림 6-23 GoogLeNet의 인셉션 모듈



- 인셉션 모듈의 네 가지 연산
 - : 1x1 합성곱, 1x1 합성곱+ 3x3 합성곱, 1x1 합성곱+5x5합성곱, 3x3최대풀링 +1x1합성곱
- 연산량이 적어 과적합 해결